Compiled landscape optimisation: exact resident state and GPU population search for harvest scheduling

UBC FRESH Lab[a]

[a] *University of British Columbia, Canada*

Abstract

Landscape forest planning models are difficult not because the objective is exotic, but because the model, the exact accounting state, and the search move are usually rebuilt together. This paper describes a compiled-state architecture that separates four concerns: a stable compiled model, compact candidate and sparse effect indexes, exact resident accounting, and batched GPU population search with exact CPU replay. A CPU-refinement handoff is designed but is not part of the measured campaign. We evaluate the architecture on a public frozen TSA24 landscape with 208,255 operable blocks. A 1,000-generation GPU search reduces the weighted penalty from 1,722,140,000 to 8,588,730 in 9.38 s of search time; a 100,000-generation, five-seed campaign reaches 512.574–518.621 (mean 515.682) with exact CPU replay. The target uses deliberately unattainable harvest-level aspirations to pull the solution toward the maximum even-flow level; the final level is approximately 51.21–52.49M m^3 per 10-year period against a 70M aspiration. A factorized 64-genome batch is 248 times faster on CUDA than on CPU with a maximum exact-evaluator error of 1.7e-6; this is batch evaluation, not an end-to-end search comparison. The frozen dataset, target, and seed are public; the paper-specific schedules and replay scripts are retained in the project evidence repository and are not yet publicly released. We do not claim an independent model comparison or a state-of-the-art landscape result; the contribution is the architecture and its reproducibility contract.

Keywords: landscape forest planning, compiled model, GPU population search, exact replay, reproducibility

Keywords: compiled spatial optimisation, harvest scheduling, exact

Email address: author@example.invalid (UBC FRESH Lab)

replay, GPU population search, reproducibility

1. Introduction

Forest planning models combine long planning horizons, spatially explicit treatment choices, and objectives that mix timber production with ecological and operational constraints. At the scale of a timber supply area, the decision is not a single harvest level but a schedule: which blocks receive which treatments, in which periods, under which spatial and temporal rules. The combinatorial state is large, the feasible set is structured, and the objective is usually a weighted compromise among targets. The resulting models are hard for exact methods and expensive for generic heuristics.

A recurring engineering problem sits underneath the optimisation problem. A compiled forest model contains blocks, fragments, strata, tracks, treatments, transitions, growth curves, products, and accounting rules. A search move changes one small part of that model, but the implementation often recomputes or reconstructs a much larger state to evaluate it. The model, the accounting state, and the move are therefore rebuilt together. This coupling makes experiments slow, makes replay fragile, and makes it difficult to separate the effect of a search method from the effect of an implementation detail.

This paper asks a narrower question than "which metaheuristic is best for forest planning?" It asks whether a compiled-state architecture can make a large landscape search exact, replayable, and fast enough to support controlled experiments. The architecture has five layers:

1. a compiled model;
2. a compact candidate index;
3. a sparse effect table over target cells;
4. exact resident accounting state;
5. a GPU population search with exact CPU replay; a CPU-refinement handoff is designed but not measured here.

The first two layers make the model cheap to query. The sparse table makes an evaluation proportional to the cells a move touches rather than to the whole schedule. The resident state makes the objective authoritative and reversible. The design loop uses the GPU for breadth and the CPU for exact accounting and local refinement; the measured campaign uses GPU search with exact replay and does not include the refinement arm.

We evaluate the architecture on a frozen TSA24/Prince George even-flow instance. The instance has 208,255 operable blocks out of 458,708

compiled blocks and a 20-period horizon. The target is a weighted goal-programming construction: a dominant even-flow target defines the shape, while deliberately unattainable harvest-level aspirations provide a persistent pull toward more volume. The realized level is an outcome, not a declared feasible floor. This construction is intentional; the 70M m^3 per-period aspiration is not a hard constraint and the reported residual is not evidence that the model is infeasible.

The paper makes three contributions:

- a compiled-state architecture that separates model, accounting, and search;
- an exact replay contract for a large landscape instance, including a public frozen artifact;
- a measured scaling record from 1,000 to 100,000 GPU generations, with five-seed evidence at the largest budget.

It does not make a competitive landscape claim. The CPU simulated-annealing comparison reported here is a weak, untuned secondary comparator, and no independent harvest-scheduling model is available on the same instance. Those limits are part of the result, not a footnote.

The unit of analysis is not a single solver run but the boundary between a compiled model and a search process. The paper asks three questions. First, can a compiled spatial model expose a stable candidate and effect representation that is rich enough for non-trivial treatment scheduling? Second, can an exact resident state price a move without re-simulating the whole schedule, while remaining identical to the full evaluator? Third, can the resulting evidence be replayed by someone who did not run the search? The answers are architectural: the compiled model is versioned, the effect table is sparse, the resident state is authoritative, and the replay gate is part of the result format.

The scope is deliberately bounded. The TSA24 case has no hard spatial constraints in the reported run. It is a landscape-scale harvest-scheduling instance with a compiled spatial structure, not a test of adjacency or opening-size feasibility. The GPU campaign uses a fixed target construction and a fixed seed set; it is not a tuned solver portfolio. The CPU-SA comparator is a weak secondary baseline. The paper therefore does not claim state-of-the-art harvest scheduling, a universal GPU speedup, or an operational recommendation.

This positioning matters for the target journal. The contribution is a modelling and software architecture with a reproducible case, not a new

metaheuristic. The evidence is strongest where the model boundary is expensive: many blocks, a long horizon, a non-separable flow target, and an exact accounting state. A reader who needs a new search operator will not find one here. A reader who needs a model-and-evidence layer that can host different operators will.

2. Related work

2.1. Spatial harvest scheduling

Spatial harvest scheduling has a long history in forest operations research. The classical formulation treats the problem as a sequence of harvest and investment decisions subject to yield, adjacency, and opening-size rules [1, 2]. **(author?)** [3] formulated a version with spatial constraints and applied simulated annealing to a problem in which adjacency and opening-size restrictions make the feasible set non-trivial. Their work established a template that remains recognizable: represent a schedule as a set of treatment decisions, define a penalty or objective over the resulting landscape, and use a stochastic search to move between feasible or penalized states.

Later work made the spatial structure more explicit. **(author?)** [4] reviewed spatial restrictions in harvest scheduling, and **(author?)** [5] surveyed spatial forest planning models and solution methods. **(author?)** [6] explored exact approaches for maximum-area restrictions, showing that spatial constraints can be handled by branch-and-bound on strengthened formulations for some instance classes. The book by **(author?)** [7] remains a useful reference for spatial optimization in managed ecosystems. These works establish both the importance of spatial rules and the difficulty of solving realistic instances exactly.

(author?) [8] compared tabu search and a genetic algorithm on spatial harvest scheduling problems. Their comparison is important here because it treats the search method as an experimental factor rather than as an implementation detail. **(author?)** [9] extended the comparison to eight heuristic planning techniques on three wildlife planning problems of increasing difficulty. Together, these studies show that the choice of neighbourhood, the representation of the schedule, and the handling of spatial constraints can matter as much as the name of the metaheuristic.

The present paper is not a replacement for those comparisons. It addresses the layer beneath them: how the model state and objective are maintained while a search runs. The architecture is designed to make that layer explicit, exact, and replayable, so that future comparisons can vary the search method without rebuilding the model and accounting machinery.

2.2. Exact state and neighbourhood search

Metaheuristic implementations often trade exactness for speed. A move is evaluated by an approximate delta, a cached value, or a full re-simulation; each choice changes what the search is actually optimising. Simulated annealing and tabu search are the canonical move-based metaheuristics [10, 11]; their modern variants are surveyed by **(author?)** [12] and **(author?)** [13]. The general design space is covered by **(author?)** [14] and **(author?)** [15], and iterated local search is a useful reference for perturbation-based extensions [16].

For forest planning, exactness is a material issue because the objective is a weighted goal-programming penalty over multiple accounts and periods. Goal programming itself has a long history in management science [17, 18, 19]. An approximate evaluator can make a schedule look better than it is, and a full re-simulation can make a controlled experiment impractically slow. Exact combinatorial formulations also become intractable quickly [20, 21], which is why heuristic search is common at landscape scale.

The architecture described here keeps the exact objective as the authority. A move is priced by the cells it changes, but the resident state is updated with the same accounting rules used by the full evaluator. The search can therefore use a fast delta without giving up the exactness of the reported incumbent. This is closest in spirit to the exact incremental evaluation used in move-based local search, but it is applied to a compiled landscape model rather than to a generic permutation problem.

2.3. Reproducibility and software infrastructure

Reproducibility in applied optimisation is usually discussed at the level of random seeds and reported objective values. That is necessary but not sufficient. A result also depends on the compiled model, the candidate order, the effect table, the treatment definitions, the objective weights, the hardware, and the replay procedure. The general reproducible-research literature makes the same point [22, 23, 24, 25]. Containers and environment capture reduce the software part of the problem [26, 27].

The experimental-methods literature in operations research is directly relevant. **(author?)** [28] argued that algorithm comparisons should test hypotheses about why a method works, not just report wins. **(author?)** [29] proposed a reporting discipline for heuristic experiments, and **(author?)** [30] developed an experimental method for algorithm simulation. More recent guidance collects best practices for comparing optimization algorithms and for benchmarking [31, 32]. This paper adopts the same posture at the level

of a single case: the evidence record, the stopping rule, and the replay gate are part of the method.

The implementation uses standard scientific Python: NumPy for array operations [33], SciPy for independent numerical baselines [34], Numba for optional acceleration [35], and PyTorch for the GPU path [36]. GPU computing has a long literature of its own; the survey by **(author?)** [37] and the CUDA programming model by **(author?)** [38] are useful entry points. The architecture described here does not depend on a particular GPU vendor, but the reported campaign does.

This paper treats the evidence record as part of the architecture. Each reported schedule is stored with its objective, seed, configuration, hardware, and a hash. An exact replay loads the schedule through the same model and evaluator, rather than trusting the search process that produced it. The frozen TSA24 artifact and its rebuild recipe are published so that the GPU result is not a local demonstration that cannot be inspected. The related work on reproducible optimisation software is broad; the narrower claim here is that an exact resident state plus an explicit artifact can make a large landscape experiment auditable even when the search itself is stochastic.

3. Problem and model

3.1. Instance

The case study uses the TSA24/Prince George landscape compiled for WS4. The compiled dataset contains 458,708 blocks; 208,255 of them have at least one operable treatment chain and therefore form the search universe. The horizon is 200 years, divided into twenty 10-year accounting periods. The model includes blocks, fragments, tracks, strata, treatments, growth and yield curves, features, products, and account definitions. The schedule assigns a treatment chain to each operable block. An empty chain is a valid decision: a block may be left untreated.

The instance is frozen and published as `ws4-tsa24-evenflow-v1` with the compiled dataset, target file, initial schedule, license terms, and a checksum manifest [39]. The artifact is a derived benchmark bundle; the raw BC source layers retain their own open-government terms.

3.2. Even-flow target and level aspirations

The objective is weighted goal programming. Each target contributes a one-sided penalty for falling below a minimum or exceeding a maximum:

$$s_{t,p} = \max(m_t - v_{t,p}, 0), \tag{1}$$
$$e_{t,p} = \max(v_{t,p} - M_t, 0), \tag{2}$$
$$\tau_{t,p} = w_t^- \, \phi(s_{t,p}) + w_t^+ \, \phi(e_{t,p}). \tag{3}$$

The shape is linear or quadratic. The target set is not a set of hard constraints. The names `minimum` and `maximum` name the hinge of the penalty, not a feasible interval.

The TSA24 target has two roles. First, one even-flow target spans periods 1–20, compares each period with the reference period, and charges a squared deviation outside a ±1% band with weight 10,000. Second, every period carries a linear harvest-level target with aspiration 70,000,000 m^3 per 10-year period and weight 100. The level aspiration is deliberately above the level the estate is expected to achieve. It is a pull-up device: every period has a downward gradient toward more harvest, including periods that would otherwise be left underused. The even-flow target selects the flattest profile at the level where the two roles reach their best weighted compromise. The realized level is therefore an outcome, not a declared floor.

This construction is the maximum-even-flow pattern documented for WS4. It is not an infeasibility error, and a large residual on the level term is not evidence that the target should be removed. It is the expected consequence of an aspirational target that creates a search gradient. The paper reports the split between the flow and level terms, the realized harvest level, and the target values so that a reader can inspect the compromise rather than infer it from a single total.

3.3. Initial schedule and feasibility

The initial schedule is the oldest-first seed from the public artifact. It is a valid block/treatment schedule and therefore a feasible starting point for the search, but it is not an optimized control. The initial objective is 1,722,140,000. The very large initial penalty is dominated by even-flow deviations; the seed is a starting point chosen for reproducibility, not a baseline chosen to make the search look effective.

The model has no hard spatial constraints in this run. The target residuals are soft, and the reported schedule is a compromise. The paper therefore distinguishes schedule validity from target satisfaction: the schedule is feasible, while the targets are aspirations.

3.4. Objective and evaluation

For a schedule `S`, let `v(S, p, a)` be the realized value of account `a` in period `p`. The objective is

$$Z(S) = \sum_{t} \sum_{p \in P_t} \tau_{t,p}(v_{t,p}(S)). \tag{4}$$

The implementation evaluates this objective exactly. A search move may use a cached per-block contribution to price its effect, but the resident state is updated with the same accounting rules as the full evaluator. The reported objective is never an approximation produced by the search loop. This distinction is central to the evidence contract: a fast delta is allowed, but the incumbent must replay through the exact evaluator.

3.5. Schedule representation and feasibility

A schedule is a total mapping from operable blocks to candidate chains. A chain is an ordered list of treatment events; the empty chain is a valid decision. The mapping is total over the operable set, so a partial schedule is not a valid search state. The candidate index guarantees that every chain in a mapping is feasible for its block under the compiled eligibility and operability rules.

Feasibility and target satisfaction are separate. A schedule is feasible when every block has one candidate chain and the chain's events obey the compiled windows. A schedule can be feasible while missing every target. The reported objective is a weighted penalty, not a feasibility measure. This distinction is important in the TSA24 case: the initial seed is feasible and has a very large objective because its flow profile is far from flat. A reader who treats the objective as a constraint violation would misread the model.

The search uses four move families: toggle a chain, shift an event in a chain, swap chains between two blocks, and destroy-and-repair a small set of blocks. The first three are local and reversible. The fourth is a compound move that is decomposed into local changes before it reaches the resident state. The evidence contract applies to the final assignment, not to the move decomposition; the replay gate evaluates the assignment, not the search history.

3.6. Target calibration

The even-flow target is calibrated on the reference period. A period below the reference by more than the band contributes a squared deviation with weight 10,000. A period above the reference by more than the band

Table 1: TSA24 target-variant screen: 1,000 SA iterations, seed 7, same start; screen only, not a converged comparison.

Target file	Objective after 1,000 iterations
`evenflow_tsa24`	1.71×10^9
`evenflow_tsa24_allperiods5`	1.57×10^9
`evenflow_tsa24_allperiods5_stock90`	1.62×10^9
`evenflow_tsa24_ltsy`	5.75×10^8
`evenflow_tsa24_ltsy_stock90`	1.01×10^3

contributes the same penalty. The band is deliberately narrow, so the target is sensitive to the shape of the profile even when the mean level is close to the reference.

The level targets are calibrated at 70M m^3 per period with weight 100. The value is above the estate's attainable level by design. The level target is therefore a pull-up gradient, not a feasibility floor. A search that treats 70M as a hard constraint will declare the instance infeasible; a search that ignores the level term will stop at a lower flat profile. The weighted construction keeps both pressures visible.

The weights are part of the evidence. A change to the even-flow weight, the level weight, or the band changes the objective and therefore changes which schedule is best. The record stores the target file and its hash so that a replay cannot silently use a different weighting. This is a modest but important form of provenance: a target configuration is data, not a constant.

A 1,000-iteration screen across the five published TSA24 target variants shows how strongly the target construction controls the objective scale. The objectives range from 1.71×10^9 for the main even-flow target to 1.01×10^3 for the LT-sy stock-90 variant under the same seed, start, and budget. This is not a converged ranking; it is a warning that a single target file is not a universal objective. The full 100,000-generation campaign is reported only for the main target, and target-weight sensitivity at the full budget remains future work.

4. Architecture

4.1. Compiled model

The compiled dataset is the boundary between ingestion and execution. It is a versioned directory of columnar tables plus a manifest. The tables describe fragments, blocks, tracks, strata, treatments, curves, features, prod-

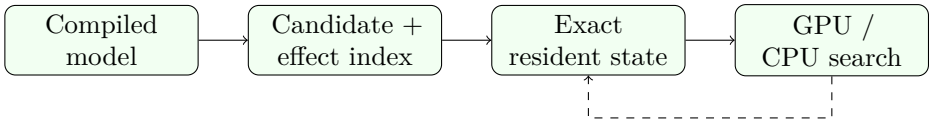


Figure 1: Compiled-state architecture. The resident state is authoritative; the search reads and writes it through exact apply/undo operations.

ucts, accounts, groups, and compile diagnostics. The manifest records the schema version, source identities and hashes, compile options, and table list.

Compilation happens at the fragment level; a block is a runtime aggregate of its fragments. The separation matters because fragments carry the inventory attributes, while the search and reporting interfaces operate on blocks. The search never reads the raw inventory or recompiles the recipe.

4.2. Candidate index

For each block, the candidate index stores the feasible treatment chains in a stable order. A chain is an ordered sequence of treatment events `(treatment, year)`. The order is part of the contract: candidate indices are referenced by the effect table and by the search, so changing the order changes the meaning of a cached effect.

The index is built by walking track transitions from the block's initial state and applying the treatment eligibility and operability windows. It includes the empty chain. The index is deliberately smaller than the full cross-product of treatments and years, but it remains richer than a single "harvest/no harvest" decision. The search can toggle a chain, shift an event, swap chains between blocks, or apply a larger destroy-and-repair move.

4.3. Sparse effect table

A compiled landscape has many blocks and many target cells, but a single block/chain pair affects only a small number of `(year, account)` cells. The effect table stores, for each block and candidate chain, a sparse row of those cells and their values relative to the untreated block. The table also stores the empty-schedule base totals and block row starts.

The representation makes evaluation local. A move on one block is priced by the cells that block's old and new chains touch, not by iterating over the whole schedule. The sparse rows are built once and cached; the table identity includes the dataset fingerprint, candidate order, target-cell geometry, horizon, period width, and table format version. A change to weights, priorities, or targets is a pricing change, not a table rebuild.

4.4. Effect-table construction and cost

The effect table is built by walking each block's candidate chains and accumulating the account cells that the chain changes relative to the untreated block. The result is a sparse row per `(block, chain)` pair. The row is not a full simulation; it is a cached contribution that can be added to the empty-schedule base. The construction is a one-time cost per compiled dataset and table format. The search pays only for the rows it touches.

The table has three costs that matter in practice. The build cost is proportional to the number of candidate chains times the number of cells per chain. The storage cost is proportional to the number of non-zero cells, not to the dense block-by-period-by-account product. The query cost for one move is proportional to the old and new rows. Those three costs are why the compiled representation is useful for a large landscape: the dense state is never materialized as a dense tensor in the search loop.

The table identity is a cache key, not a comment. If the dataset fingerprint changes, the table is rebuilt. If only the target weights change, the table is reused and the pricing layer is recomputed. That separation keeps an experiment from silently mixing a new model with an old cache. The manifest records the identity, and the replay path checks the schedule against the same compiled model.

4.5. Resident-state update invariant

The resident state stores realized totals per `(period, account)` plus the current chain for each block. Applying a move subtracts the old row and adds the new row. Undo applies the inverse. The invariant is that the state is always the exact sum of the empty-schedule base and the rows of the current assignment. The invariant is checked by the replay path, which reconstructs the state from the assignment and compares the objective.

The update is not allowed to depend on the move history. A block that has been toggled many times must have the same row contribution as a block that reached the same chain in one step. This is what makes the state reversible and replayable. It also means that a compound move must be decomposed into elementary changes before it is applied; the resident state never stores a search-specific diff.

The invariant has a cost. The state must be updated even when the search only needs a comparison, and the exact objective must be recomputed when the incumbent changes. The evaluation-path ablation measures that cost on a 5,000-block subset; it is the reason the paper reports a diagnostic rather than a full-instance ablation.

4.6. Batched evaluation and exact replay

The GPU population evaluates a batch of candidate-index genomes against the sparse table. The batch is a matrix of block/chain choices; the kernel accumulates the corresponding rows into a per-genome objective. The GPU path is a search accelerator, not the evidence authority. The CPU replay path reconstructs the resident state from the stored assignment and computes the objective with the same accounting rules.

The two paths can differ in floating-point order. The record therefore stores the CPU replay objective, not the device objective, and the replay gate compares the stored schedule with the CPU evaluator. A small device/CPU difference is acceptable as a search signal; it is not acceptable as a reported result. This separation is deliberate: the paper reports what can be replayed, even when the search used a faster device path.

The GPU path also imposes a memory budget. The effect table must fit in device memory alongside the population batch. The TSA24 table is stored in a lean format that omits the full row index; the population uses row starts. That choice is an implementation detail, but it is part of the architecture: a table that does not fit cannot be used for a population search, regardless of its mathematical elegance.

4.7. Failure modes

The architecture has four failure modes that the evidence contract is designed to expose. A model change without a table rebuild produces a cache mismatch. A move that is not reversible produces a replay mismatch. A search that reports the device objective produces a record that does not match the CPU replay. A schedule that is partial or infeasible fails the adapter's validation. None of these failures is silent in a record that passes the gate.

The gate does not catch a bad search design. A search can converge to a poor schedule and still produce an exact, replayable record. The paper reports the objective distribution and the target residuals so that a reader can judge the search, not just the record. That is the division of responsibility: the harness guarantees the evidence, and the paper argues about the search.

4.8. Resident exact state

The optimizer maintains a resident state of realized totals per `(year, account)`, together with the current chain assigned to each block. Applying a move adds the new chain's contribution and subtracts the old chain's contribution. Undo restores the previous state exactly. The state is authoritative:

the search uses it for the objective, and the replay path reconstructs it from the schedule through the same evaluator.

For a move on block `b` from chain `old` to chain `new`, let p be the sparse pending change to the realized account totals and let $\mathcal{T}$ be the targets it affects. The objective delta is

$$\Delta Z_b(c_{\text{old}} \to c_{\text{new}}) = \sum_{t \in \mathcal{T}} \left[\pi_t(T + p) - \pi_t(T) \right], \tag{5}$$

where π_t is the target's penalty function. For a linear level target, π_t is a per-cell penalty. For the even-flow target, π_t is a penalty over the period vector, so the delta re-evaluates the affected flow target rather than summing independent cell penalties. The cost is proportional to the pending cells plus the flow target's period vector, not to the number of blocks in the schedule. The implementation uses this identity in the local-search paths and retains a whole-schedule evaluator as a reference.

4.9. GPU population search and exact replay

The architecture separates breadth from exactness. A batched GPU population evaluates candidate-index genomes against the sparse effect table. The GPU path is used for population-scale exploration; it does not replace the exact accounting state.

The design also supports a CPU-refinement handoff: load an elite schedule into the exact resident state, apply local moves, and write the incumbent back as a GPU seed. That handoff is a design path, not a measured contribution in this paper. The 100,000-generation campaign reported below is a `ws4-gpu-ga` run with exact CPU replay; it does not contain a CPU-refinement arm or CPU-refinement counters. The handoff is therefore shown as a possible extension, not as evidence for the scaling result.

The measured division is narrower: GPU population search produces the incumbents, and the exact CPU evaluator is the replay authority. This is an engineering choice, not a claim that GPU search is universally better than CPU search.

4.10. Replay and provenance

Every reported schedule is stored as a complete block/treatment/offset assignment. The record carries the objective, seed, configuration, hardware, software, schedule path, and SHA-256. Replay loads the schedule through the compiled model and exact evaluator and compares the resulting objective with the record. A record with a mismatched hash or objective is not evidence.

The architecture therefore has two levels of reproducibility. The search is reproducible from its configuration and seed for a fixed implementation, and the reported result is independently replayable from the stored schedule even if the search process is not rerun. The second property is the one the paper relies on for its tables.

The record format is intentionally explicit. A record names the instance and its availability, the objective and feasibility definition, the algorithm, version and configuration, the seed, the hardware and software, the wall and search times, the schedule path and hash, the exact-replay flag, and the per-target residuals. The fields are not all free text: a validator checks the schema, the schedule path, and the hash. The reported result is the replayed objective, not a value copied from a log.

The replay gate is a separate program from the search. It loads the compiled model, reads the stored schedule, validates the assignment, and evaluates the objective. It does not call the optimizer. A record can therefore pass the gate even if the optimizer binary is unavailable, which is the point of storing the assignment rather than only the seed. The gate also checks that the schedule covers the operable block set and that every chain is a candidate for its block.

The gate has limits. It verifies the objective for the stored schedule, not the search trajectory. It does not prove that the search explored the neighbourhood it claims, and it does not prove that the seed was used as stated. The artifact contains the configuration and the replay script so that a reader can inspect those claims, but the formal guarantee is narrower: the incumbent is exact and replayable. The paper uses that guarantee and does not inflate it.

The record format is also a review tool. A table can be generated from the records, and a stale number can be detected by regenerating the table. The paper's evidence table is generated from the record files, not typed by hand. This is a small practice, but it prevents the most common reproducibility defect: a result that is correct in the log and wrong in the manuscript.

4.11. Instance and target

All experiments use the frozen TSA24/Prince George artifact described in Section 3. The compiled dataset contains 458,708 blocks; 208,255 are operable and form the search universe. The horizon is 200 years, divided into twenty 10-year periods. The target combines a weight-10,000 even-flow band with weight-100 level aspirations of 70,000,000 m^3 per period. The level aspiration is deliberately unattainable. It is a pull-up gradient, not a feasibility constraint.

4.12. Hardware and software

The GPU runs used an NVIDIA RTX PRO 6000 Blackwell Max-Q Workstation Edition (97,887 MiB) with an Intel Xeon Gold 6254 CPU at 3.10 GHz. The software was Python 3.12.3, PyTorch, and WS4 [40] at commit `f60ce64` on the `feature/gpu-acceleration` branch. The CPU baseline used WS4 at commit `f23d450` with 32 threads; the record does not claim that commit is on main. The public artifact and its rebuild recipe are released separately; the large GPU effect table and caches are not part of the 4.66 MB artifact.

4.13. Search configurations

The GPU population size was 64. We report three budgets: 1,000, 10,000, and 100,000 generations. The 1,000- and 10,000-generation runs use seed 7 and correspond to the earlier architecture and scaling records. The 100,000-generation campaign uses seeds 1–5; seed 7 is retained as a reference run. Every GPU run starts from the same oldest-first feasible seed. The initial objective is 1,722,140,000.

The CPU baseline is simulated annealing at 100,000 iterations with a nominal 30-minute limit and 32 threads, using the same instance, target, and seed. It is a secondary, untuned comparator. It performs 100,000 single proposals, whereas the GPU-GA path evaluates a population of 64 for 100,000 generations (6.4 million genome evaluations). The two budgets are therefore not equal-work or equal-time, and the comparison is reported only to show the scale of the gap.

4.14. Metrics and replay

The primary metric is the weighted objective. We also report the flow and level components separately, the realized harvest-level range, search time, wall time, and seed distribution. Every reported schedule is replayed through the exact CPU `IncrementalObjective`; the recorded objective is the replay objective, not the device objective. A record with a mismatched schedule hash or replay objective is not evidence.

A full-instance whole-schedule ablation is impractical: a single full evaluation of the 208,255-operable-block schedule is the motivating cost. We therefore report a disclosed diagnostic ablation on the first 5,000 blocks of the recorded 100k schedule. The subset is not a substitute for a full-instance ablation, but it isolates the evaluation path from the GPU population search.

Numeric reporting follows one rule. Counts, seeds, iterations, and blocks are exact. Campaign objectives and flow/level terms use six significant

Table 2: Seed-7 scaling on the frozen TSA24 instance. Wall time includes model and cache setup; search time is the GPU search budget.

Generations	Objective	Flow term	Level term	Search / wall (s)
1,000	8,588,730	8,588,230	499.821	9.38 / 23.1
10,000	76,540.7	76,026.9	513.832	89.8 / 157
100,000	517.862	2.721	515.141	856 / 942

figures; screen and one-seed diagnostic objectives use three. Optimality gaps use three decimal places in percentage points. Times, ratios, and CPU overheads use three significant figures. Full-precision values remain in the evidence files.

5. Results

5.1. Scaling from 1,000 to 100,000 generations

Table 2 reports the seed-7 runs at the three budgets. The 1,000-generation run reduced the objective from 1,722,140,000 to 8,588,730 in 9.38 s of search time (23.1 s wall). The 10,000-generation run reached 76,540.7 in 89.8 s search (157 s wall). The 100,000-generation run reached 517.862 in 856 s search (942 s wall). The 100,000-generation result is not a near-zero objective: the flow term is small, but the deliberate level-aspiration term remains.

The shape of the scaling curve is informative. The objective falls by roughly two orders of magnitude from 1,000 to 10,000 generations and by another two orders from 10,000 to 100,000. That is not a convergence proof; it is a statement that the search is still making progress at the largest budget. The flow term collapses much faster than the level term, because the even-flow target is the part of the objective that the population can improve by reshaping the profile. The level term remains high because the target is above the achievable level, not because the search is ignoring it.

The curve also shows why short-budget claims are risky. A 1,000-generation run is a useful smoke test and a useful scaling anchor, but it is not a competitive result. A 10,000-generation run is closer, yet the 100,000-generation run is still materially better. The paper reports all three so that the reader can see the budget sensitivity rather than only the final number.

5.2. Five-seed 100,000-generation campaign

Table 3 reports the five-seed campaign at 100,000 generations. The objectives are tightly clustered: the best seed is 512.574, the worst is 518.621,

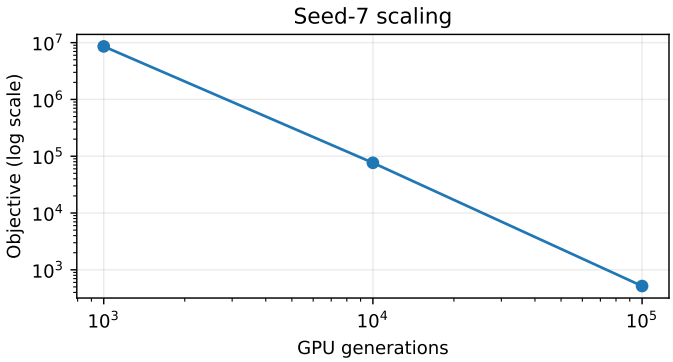


Figure 2: Seed-7 scaling from 1,000 to 100,000 GPU generations. Both axes are logarithmic.

Table 3: Five-seed 100,000-generation campaign. All objectives are exact CPU replays.

Seed	Objective	Flow term	Level term	Realized level (M m^3/period)
1	518.621	1.331	517.290	51.21–52.25
2	512.574	0.000	512.574	51.46–52.49
3	515.938	0.000	515.938	51.31–52.32
4	514.746	0.000	514.746	51.34–52.35
5	516.530	0.076	516.454	51.26–52.29
7	517.862	2.721	515.141	51.34–52.38

the mean is 515.682, and the median is 515.938. Seed 7, at 517.862, lies inside the same range. All five schedules replay exactly. The campaign took 4.00×10^3 s wall for all five runs.

The realized level is approximately 51.21–52.49M m^3 per 10-year period against the 70M aspiration. The level term is therefore the dominant part of the final objective, not a residual that the search failed to remove. The even-flow term falls from 8.58823M at 1,000 generations to 2.721 at 100,000 generations; the level term remains near 499.821–517.290 because the target is deliberately above the achievable level. This is the expected maximum-even-flow behaviour of the target construction.

The distribution is tight in objective but not identical in residuals. Some seeds reach a lower flow term and a slightly higher level term; others do the reverse. The five-seed spread is therefore not noise around a single schedule, but a small set of different compromises. The replay evidence records the full assignment for each seed so that the compromise can be inspected. The paper reports the mean and median, but the distribution is the result: a single seed could make the level/flow trade-off look cleaner than it is.

The residual split also explains why the total objective can look small relative to the initial penalty while the level term remains in the hundreds. The initial seed is dominated by flow deviations; the final schedules have almost no flow penalty but still pay for the level aspiration. A reader who sees only the total might infer that the target was nearly satisfied. The split prevents that inference.

5.3. CPU simulated-annealing comparator

The CPU simulated-annealing run reached 3.35045×10^8 in 1.88×10^3 s wall (31.3 min) and stopped at the nominal 30-minute limit after 100,000 iterations. It is not converged and not a controlled baseline. Its objective is far above the GPU-GA results in Table 2. We do not interpret this as a universal architecture win: the CPU run is untuned, single-seed, and performs much less work than the GPU population. It is reported to keep the comparison honest and to show that the CPU-SA path is not a substitute for the accelerated search in this setting.

The comparator is deliberately not normalized. The CPU-SA run has 204,413 operable blocks in its schedule, while the GPU schedule has 208,255. The difference is small relative to the instance, but it means the two runs do not cover exactly the same decision set. The CPU run also uses 100,000 proposals, while the GPU run evaluates a population of 64 for 100,000 generations. The paper reports the comparison as a scale illustration, not as an equal-work or equal-time result. A reader who wants a controlled CPU/GPU comparison should treat this table as a motivation for the architecture ablation, not as the ablation itself.

The CPU-SA result is still useful as a negative control. It shows that a generic single-proposal search, given a modest budget and an untuned annealing schedule, does not approach the GPU population result on this instance. That is not surprising, and it is not a claim that CPU search is inherently worse. It is evidence that the architecture's search layer matters, while leaving the causal attribution open.

5.4. Evaluation-path ablation

On the 5,000-block diagnostic subset, a cold whole-schedule evaluation took 54.9 s; after warm-up, a whole-schedule move evaluation took 1.09 s. Exact incremental delta plus apply took 0.542 ms per move, a factor of 2.00×10^3. The final incremental objective matched the whole-schedule objective to 3.7×10^{-9}. The subset scope is recorded in `evidence/ws4-tsa24-eval-ablation.json`. This establishes the local evaluation-path effect; it does not measure a matched CPU-GA/search or

Table 4: TSA24 factorized mixed feature/product batch ablation. Same 64 genomes and four cells on both devices; batch evaluation only, not end-to-end search.

Device	Batch (s)	Genomes/s	Peak GPU (GB)	Max error
CPU	23.1	2.77	–	1.71×10^{-6}
CUDA	0.0931	687	9.27	1.71×10^{-6}

establish full-instance scaling. The factorized 64-genome batch in the next subsection is a separate microbenchmark.

The ablation isolates the evaluation path, not the GPU. On the 5,000-block subset, a warm whole-schedule move evaluation takes about 1.09 s, while the exact delta plus apply takes about 0.542 ms. The ratio is about 2.00×10^3. The exactness check is the important part: the final incremental objective matches the whole-schedule objective to 3.7e-9. The result supports the claim that the sparse effect table plus resident state can avoid a full re-simulation without changing the objective. It does not support a claim that the GPU population is faster than a matched CPU search; that experiment remains open.

5.5. Factorized batch ablation

The factorized-effect benchmark is a second, independent architecture check. It evaluates the same 64-genome batch against the same four mixed feature/product cells on CPU and CUDA, using the TSA24 factorized archive. The CPU batch takes 23.1 s; the CUDA batch takes 0.0931 s, a factor of 248. The throughput is 2.77 genomes/s on CPU and 687 genomes/s on CUDA. The maximum absolute error against the exact evaluator is 1.71×10^{-6} on both paths. The factorized build takes 234 s and the GPU peak allocation is 9.27 GB.

This is a real GPU-versus-CPU batch effect, but it is not an end-to-end search comparison. The CPU path is a one-device tensor path, not a tuned multithreaded batch; the result supports the factorized evaluation path, not a universal architecture claim. The 234 s build cost is also part of the result: the batch advantage matters only when the table is reused. The next experiment is a matched CPU-GA at the same population/generation count, which remains open.

5.6. Matched implementation search

The same `gpu-run` implementation accepts `-device cpu` and `-device cuda`. We ran it on the full TSA24 model with population 64, seed 7, and

Table 5: TSA24 matched implementation search: full model, population 64, seed 7, one seed. CPU is a single torch device path, not a tuned CPU-GA. The 100-generation CUDA run uses 22-fold less search time, but its objective is higher; this is not a hardware claim.

Generations	CPU search (s)	CUDA search (s)	CPU/CUDA time	CPU objective	CUDA objective
1	26.2	41.7	0.63	3.18×10^8	3.47×10^8
10	71.5	33.7	2.12	1.23×10^8	1.34×10^8
100	529	23.9	22.2	7.11×10^7	7.49×10^7

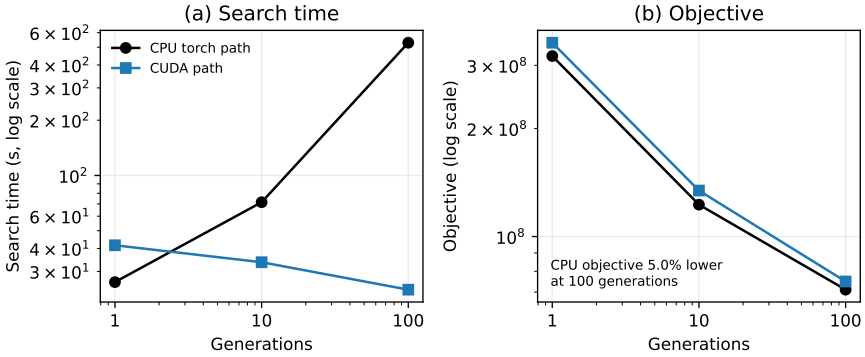


Figure 3: Matched CPU/CUDA implementation search on the full TSA24 model, population 64, seed 7. (a) Search time; the CUDA path is slower at one generation but 22.2 times shorter at 100 generations. (b) Objective; the CPU path remains 5% lower at 100 generations. This is a short-horizon implementation comparison, not a hardware ranking.

1, 10, and 100 generations. The CPU path is one torch device path, not a tuned multithreaded CPU-GA; each row is one seed. At one generation the CUDA path is slower (41.7 s vs 26.2 s search), because model/table overhead dominates. At 100 generations CUDA search is 23.9 s vs 529 s on CPU, a 22.2-fold shorter search time in this one-seed run, but the CPU objective is about 5% lower (7.11×10^7 vs 7.49×10^7). This is a short-horizon matched implementation comparison, not a quality ranking, not a tuned CPU baseline, and not a full 100,000-generation CPU comparison. It also shows that the observed time ratio is generation-count dependent; it is not a hardware claim.

5.7. What the results establish

The results establish four things. First, the compiled-state pipeline can run a 208,255-operable-block instance at a 100,000-generation GPU budget and produce an exact, replayable schedule. Second, the seed distribution at that budget is tight on this instance. Third, the maximum-even-flow construction produces a flat profile at a level substantially below the aspirational

70M target; the remaining level penalty is intentional. Fourth, the CPU-SA comparator is weak and does not support a controlled performance claim.

A fifth result is negative. The reported campaign does not establish that the GPU path is faster per unit of exact work than a tuned CPU implementation, and it does not establish that the architecture is the cause of the observed quality. Those are not hidden limitations; they are the reason the paper reports the evaluation-path ablation separately from the GPU campaign. The ablation isolates the cost of a whole-schedule evaluation, not the benefit of the GPU. The GPU result is a scale demonstration on one landscape, not a controlled architecture comparison.

The results do not establish that the GPU path is faster per unit of exact work than a tuned CPU implementation, that the architecture is the cause of the observed quality, or that the final schedule is optimal. Those questions require the architecture ablation and matched CPU-GA/CPU-SA experiments described in the limitations.

6. Discussion

The architecture addresses a practical failure mode in applied landscape optimisation: the model, the accounting state, and the search move are often rebuilt together. Separating them has three consequences. The compiled dataset can be versioned and reused. A move can be priced exactly from a sparse effect row. The resident state can be replayed independently of the search process. These properties are valuable even when the search itself is a standard metaheuristic.

The maximum-even-flow construction is a good example of why exact accounting matters. The objective is not a single harvest target. It is a weighted compromise between a shape target and deliberately unattainable level aspirations. A search that treats the level target as a hard floor would misread the model; a search that ignores the level term would stop at a lower flat profile. The target design requires both terms to remain visible. The reported residual table is therefore part of the result, not an implementation diagnostic.

The GPU/CPU handoff is an engineering compromise. The GPU path provides population-scale exploration; the CPU path provides exact state and local refinement. The present experiments show that the combined path can reach a tight seed distribution at 100,000 generations on the TSA24 instance. They do not isolate the contribution of the GPU search from the contribution of the compiled state or the target construction. That isolation is the next experimental step.

6.1. Threats to validity

The first threat is internal validity. The GPU campaign varies the seed and the generation budget, but it does not vary the architecture, the target construction, or the model compilation. A difference between the GPU and CPU paths therefore cannot be attributed to the GPU alone. The paper reports the scaling curve and the CPU-SA comparator as observations, not as a controlled decomposition. The evaluation-path ablation is a controlled microbenchmark on a subset, but it is not a full-instance ablation.

The second threat is external validity. TSA24 is one landscape, one target construction, and one 20-period horizon. The architecture may behave differently on a model with hard adjacency constraints, a different treatment vocabulary, or a much larger candidate set. The paper does not claim transferability to all forest planning models; it claims that the architecture is explicit and replayable on this case, and that the case is large enough to exercise the compiled-state boundary.

The third threat is construct validity. The objective is a weighted goal-programming penalty, not a timber-supply value or an ecological outcome. A schedule with a low penalty is a schedule that satisfies the target construction well; it is not necessarily a schedule that a manager would choose. The paper reports the level/flow split and the realized level so that the construct is inspectable, but it does not convert the objective into an operational recommendation.

The fourth threat is statistical. The five-seed campaign is a small sample. The objectives are tightly clustered, but the paper does not report a confidence interval for the mean or a test against a baseline. The seed distribution is descriptive. A larger seed set would be needed to estimate the tail of the search distribution, especially on a different landscape.

6.2. Operational implications

The architecture is useful when the model boundary is the expensive part of the project. A team that already has a compiled model and an exact evaluator can add a candidate index and a sparse effect table without changing the model semantics. A team that has a fast heuristic but no replay contract can wrap the heuristic in the adapter and keep the evidence boundary. A team that is comparing search methods can hold the model and target fixed and vary only the search layer.

The architecture is not a substitute for model validation. The target construction is a policy choice; the architecture can make it exact and replayable, but it cannot make it the right policy. The TSA24 case is a

scenario configuration, not an official decision. The paper's operational claim is therefore limited to the reproducibility of the computation, not the desirability of the result.

The evidence contract is also useful for review. A reviewer can replay a schedule without rerunning the search, inspect the target residuals, and compare the objective with the stored record. That is a lower bar than full computational reproducibility, but it is a higher bar than a table of numbers. The paper treats it as the minimum for a large landscape result.

A practical checklist follows from the architecture. Freeze the compiled model and record its fingerprint. Freeze the candidate order and the target file. Store the assignment, not just the seed. Replay the assignment through a separate evaluator. Report the stop reason and the budget. Keep the hardware and software versions with the campaign. None of these steps is expensive relative to the search, and each closes a failure mode that a reader cannot otherwise detect. The checklist is not a claim that every experiment will be reproducible; it is the minimum that makes the reported incumbent inspectable.

6.3. Future work

The matched implementation comparison below is a start, but a tuned CPU batched evaluator, a cache-load/search split, a full effect-table-versus-direct-evaluator comparison, and a matched CPU-GA/CPU-SA at equal work and wall time remain. Those are controlled comparisons; the current CPU-SA run is not.

The second direction is a spatial-constraint extension. TSA24 has no hard adjacency or opening-size rules in the reported run. Adding them would test whether the candidate index and effect table remain compact when feasibility depends on neighbouring blocks. That is a modelling contribution, not just an implementation change.

The third direction is a wider evidence release. The public TSA24 artifact makes the dataset and target inspectable; the paper-specific schedules and replay scripts are the remaining gap. A public release with a DOI and a one-command replay would turn the current evidence into an archival artifact. That step requires maintainer approval and is not claimed here.

7. Limitations and conclusion

The TSA24 instance is a public frozen artifact derived from public BC data, but there is no independent harvest-scheduling model baseline on the

same instance. The paper cannot claim competitive landscape performance, state-of-the-art quality, or partner-endorsed operational impact.

The 100,000-generation result is a maximum-even-flow compromise, not a proof of the exact maximum. The 70M level aspiration is deliberately unattainable; the realized level is approximately 51.21–52.49M m^3 per period. An exact MIP proof is not practical at this scale with the available resources.

The CPU-SA comparator is untuned, single-seed, time-limited, and not equal-work. It is not a controlled baseline. The evaluation-path ablation is diagnostic rather than full-instance: it uses a 5,000-block subset. The factorized 64-genome batch is a separate microbenchmark, not an end-to-end matched CPU-GA/search comparison. The short-horizon CPU/CUDA implementation comparison is reported, but the CPU path is not a tuned multithreaded baseline. A tuned CPU-GA and cache/search ablation at equal work are still required before the architecture claim can be separated from a local implementation-speed claim.

Within those limits, the paper supports a narrower conclusion: a compiled-state architecture can make a 208,255-block landscape search exact, replayable, and fast enough for a 100,000-generation GPU campaign, and a public artifact makes the result independently inspectable. That is the contribution. The competitive landscape claim remains open.

The practical lesson is that the evidence boundary should be designed with the model, not added after the search. A record that stores the schedule, the exact objective, the target residuals, and the hardware is more useful than a larger table of runs that cannot be replayed. The architecture in this paper is one way to make that boundary explicit. It is not the only way, and it is not a substitute for a controlled comparison. It is a foundation on which such a comparison can be built without rebuilding the model and accounting layer for every experiment.

The remaining work is empirical, not rhetorical. A tuned CPU-GA, a full-campaign CPU comparison, a cache/search split, and a public evidence release would decide whether the architecture claim is a speed claim or only a reproducibility claim. Until those experiments exist, the paper should be read as a careful case study of the boundary. That is a useful result, and it is the one the evidence supports.

A reader who wants a competitive solver should wait for the controlled comparison. A reader who wants an exact, replayable large-landscape case study can use the artifact now. The distinction is the paper's contribution and its limit. The evidence is deliberately presented as a reproducible case, not as a leaderboard entry. A benchmark entry would require a matched baseline and a public schedule release; neither is claimed here. The case is

still useful because the model and the replay path are inspectable. That is the minimum for a case study, and it is what the artifact provides. A reader can disagree with the search and still verify the incumbent. That is the contract. It is narrow, testable, and bounded for review now.

Data and code availability

The TSA24 benchmark artifact is available at `https://github.com/UBC-FRESH/femic-tsa24-instance/releases/tag/ws4-tsa24-evenflow-v1`. The build and replay recipe is in `BENCHMARK_ARTIFACT.md` in that repository. The paper-specific schedules and replay scripts are retained in the project evidence repository; a public evidence release remains pending maintainer approval.

References

[1] K. N. Johnson, H. L. Scheurman, Techniques for Prescribing Optimal Timber Harvest and Investment under Different Objectives: Discussion and Synthesis, Vol. 18 of Forest Science Monograph, Society of American Foresters, 1977.

[2] L. S. Davis, K. N. Johnson, P. Bettinger, T. E. Howard, Forest Management: To Sustain Ecological, Economic, and Social Values, 4th Edition, McGraw-Hill, 2001.

[3] C. Lockwood, T. Moore, Harvest scheduling with spatial constraints: a simulated annealing approach, Canadian Journal of Forest Research 23 (1993) 468–478. doi:10.1139/x93-065.

[4] A. T. Murray, Spatial restrictions in harvest scheduling, Forest Science 45 (1) (1999) 45–52.

[5] E. Z. Başkent, S. Keles, Spatial forest planning: A review, Ecological Modelling 188 (2–4) (2005) 145–173. doi:10.1016/j.ecolmodel.2005.02.018.

[6] M. Goycoolea, A. T. Murray, F. Barahona, R. Epstein, A. Weintraub, Harvest scheduling subject to maximum area restrictions: Exploring exact approaches, Operations Research 53 (3) (2005) 490–500.

[7] J. G. Hof, M. Bevers, Spatial Optimization for Managed Ecosystems, Columbia University Press, 1998.

[8] K. Boston, P. Bettinger, Combining tabu search and genetic algorithm heuristic techniques to solve spatial harvest scheduling problems, Forest Science 48 (1) (2002) 35–46. doi:10.1093/forestscience/48.1.35.

[9] P. Bettinger, D. Graetz, J. Sessions, et al., Eight heuristic planning techniques applied to three increasingly difficult wildlife planning problems, Silva Fennica 36 (2002). doi:10.14214/sf.545.

[10] S. Kirkpatrick, C. D. Gelatt, M. P. Vecchi, Optimization by simulated annealing, Science 220 (4598) (1983) 671–680. doi:10.1126/science.220.4598.671.

[11] F. Glover, Tabu search—part i, ORSA Journal on Computing 1 (3) (1989) 190–206. doi:10.1287/ijoc.1.3.190.

[12] C. Blum, A. Roli, Metaheuristics in combinatorial optimization: Overview and conceptual comparison, ACM Computing Surveys 35 (3) (2003) 268–308. doi:10.1145/937503.937505.

[13] I. Boussaïd, J. Lepagnot, P. Siarry, A survey on optimization metaheuristics, Information Sciences 237 (2013) 82–117. doi:10.1016/j.ins.2013.02.015.

[14] H. H. Hoos, T. Stützle, Stochastic Local Search: Foundations and Applications, Morgan Kaufmann, 2004.

[15] E.-G. Talbi, Metaheuristics: From Design to Implementation, Wiley, 2009.

[16] H. R. Lourenço, O. C. Martin, T. Stützle, Iterated local search, in: F. Glover, G. Kochenberger (Eds.), Handbook of Metaheuristics, Kluwer, 2003, pp. 320–353.

[17] A. Charnes, W. W. Cooper, Management Models and Industrial Applications of Linear Programming, Wiley, 1961.

[18] J. P. Ignizio, Goal Programming and Extensions, Lexington Books, 1976.

[19] C. Romero, Handbook of Critical Issues in Goal Programming, Pergamon Press, 1991.

[20] M. R. Garey, D. S. Johnson, Computers and Intractability: A Guide to the Theory of NP-Completeness, W. H. Freeman, 1979.

[21] L. A. Wolsey, Integer Programming, Wiley, 1998.

[22] R. D. Peng, Reproducible research in computational science, Science 334 (6060) (2011) 1226–1227. doi:10.1126/science.1213847.

[23] G. K. Sandve, A. Nekrutenko, J. Taylor, E. Hovig, Ten simple rules for reproducible computational research, PLoS Computational Biology 9 (10) (2013) e1003285. doi:10.1371/journal.pcbi.1003285.

[24] G. Wilson, D. A. Aruliah, C. T. Brown, N. P. Chue Hong, M. Davis, R. T. Guy, S. H. D. Haddock, K. D. Huff, I. M. Mitchell, M. D. Plumbley, B. Waugh, E. P. White, P. Wilson, Best practices for scientific computing, PLoS Biology 12 (1) (2014) e1001745. doi:10.1371/journal.pbio.1001745.

[25] V. Stodden, M. McNutt, D. H. Bailey, E. Deelman, Y. Gil, B. Hanson, M. A. Heroux, J. P. A. Ioannidis, M. Taufer, Enhancing reproducibility for computational methods, Science 354 (6317) (2016) 1240–1241. doi:10.1126/science.aah6168.

[26] C. Boettiger, An introduction to docker for reproducible research, ACM SIGOPS Operating Systems Review 49 (1) (2015) 71–79. doi:10.1145/2723872.2723882.

[27] G. M. Kurtzer, V. Sochat, M. W. Bauer, Singularity: Scientific containers for mobility of compute, PLoS ONE 12 (5) (2017) e0177459. doi:10.1371/journal.pone.0177459.

[28] J. N. Hooker, Testing heuristics: We have it all wrong, Journal of Heuristics 1 (1) (1995) 33–42. doi:10.1007/BF02430364.

[29] R. S. Barr, B. L. Golden, J. P. Kelly, M. G. C. Resende, W. R. Stewart, Designing and reporting computational experiments with heuristic methods, INFORMS Journal on Computing 7 (1) (1995) 1–17.

[30] C. C. McGeoch, Toward an experimental method for algorithm simulation, INFORMS Journal on Computing 8 (1) (1996) 1–15. doi:10.1287/ijoc.8.1.1.

[31] V. Beiranvand, W. Hare, Y. Lucet, Best practices for comparing optimization algorithms, Optimization and Engineering 18 (4) (2017) 815–848. doi:10.1007/s11081-017-9366-1.

[32] T. Bartz-Beielstein, C. Doerr, D. van den Berg, J. Bossek, S. Chandrasekaran, T. Eftimov, A. Fischbach, P. Kerschke, W. La Cava, M. Lopez-Ibanez, K. M. Malan, J. H. Moore, B. Naujoks, P. Orzechowski, V. Volz, M. Wagner, T. Weise, Benchmarking in optimization: Best practice and open issues (2020). arXiv:2007.03481.

[33] C. R. Harris, K. J. Millman, S. J. van der Walt, R. Gommers, P. Virtanen, D. Cournapeau, E. Wieser, J. Taylor, S. Berg, N. J. Smith, R. Kern, M. Picus, S. Hoyer, M. H. van Kerkwijk, M. Brett, A. Haldane, J. F. del Río, M. Wiebe, P. Peterson, P. Gérard-Marchant, K. Sheppard, T. Reddy, W. Weckesser, H. Abbasi, C. Gohlke, T. E. Oliphant, Array programming with NumPy, Nature 585 (7825) (2020) 357–362. doi:10.1038/s41586-020-2649-2.

[34] P. Virtanen, R. Gommers, T. E. Oliphant, et al., SciPy 1.0: fundamental algorithms for scientific computing in Python, Nature Methods 17 (2020) 261–272. doi:10.1038/s41592-019-0686-2.

[35] S. K. Lam, A. Pitrou, S. Seibert, Numba: a LLVM-based Python JIT compiler, in: Proceedings of the Second Workshop on the LLVM Compiler Infrastructure in HPC, 2015, pp. 1–6. doi:10.1145/2833157.2833162.

[36] A. Paszke, S. Gross, F. Massa, A. Lerer, J. Bradbury, G. Chanan, T. Killeen, Z. Lin, N. Gimelshein, L. Antiga, A. Desmaison, A. Köpf, E. Yang, Z. DeVito, M. Raison, A. Tejani, S. Chilamkurthy, B. Steiner, L. Fang, J. Bai, S. Chintala, PyTorch: An imperative style, high-performance deep learning library, in: Advances in Neural Information Processing Systems, Vol. 32, 2019, pp. 8024–8035.

[37] J. D. Owens, D. Luebke, N. Govindaraju, M. Harris, J. Krüger, A. E. Lefohn, T. J. Purcell, A survey of general-purpose computation on graphics hardware, Computer Graphics Forum 27 (1) (2008) 80–112. doi:10.1111/j.1467-8659.2008.01129.x.

[38] J. Nickolls, I. Buck, M. Garland, K. Skadron, Scalable parallel programming with CUDA, Queue 6 (2) (2008) 40–53. doi:10.1145/1365490.1365500.

[39] UBC FRESH Lab, TSA24 even-flow WS4 benchmark artifact, version 1, https://github.com/UBC-FRESH/femic-tsa24-instance/releases/tag/ws4-tsa24-evenflow-v1 (2026).

[40] UBC FRESH Lab, WS4: Wood supply simulation system, https://github.com/UBC-FRESH/ws4, gPU evidence commit f60ce64 (2026).